

Augura

One deployment. Seven autonomous modules.

The architecture of Augura's production backend: a modular Python/FastAPI monolith deployed on Modal, Supabase as the single data gateway, and one type contract that runs from SQL to the front end.

The value is proven. Now build it to last.

The current backend is an unacknowledged fracture. Eleven Vercel serverless functions in JavaScript carry the LLM agents and the proxies, while the statistical engine (bootstrap $N = 1,000$ with ATE, ATT, LME and IPW estimators) lives in Python scripts detached from the API. Two runtimes, two type systems, no bridge.

The limits are reached: the profiling agent hits the 300 second timeout, there is no mechanism for long jobs or cron, RLS is disabled on the tables, workflow state lives in the browser's sessionStorage, and demo mode is entangled with production code.

Each new capability costs more than the last.

Two runtimes by accident. One by decision.

DECISION

A modular Python/FastAPI monolith deployed on Modal. Supabase, on a fresh instance, as database, auth and storage.

SCOPE

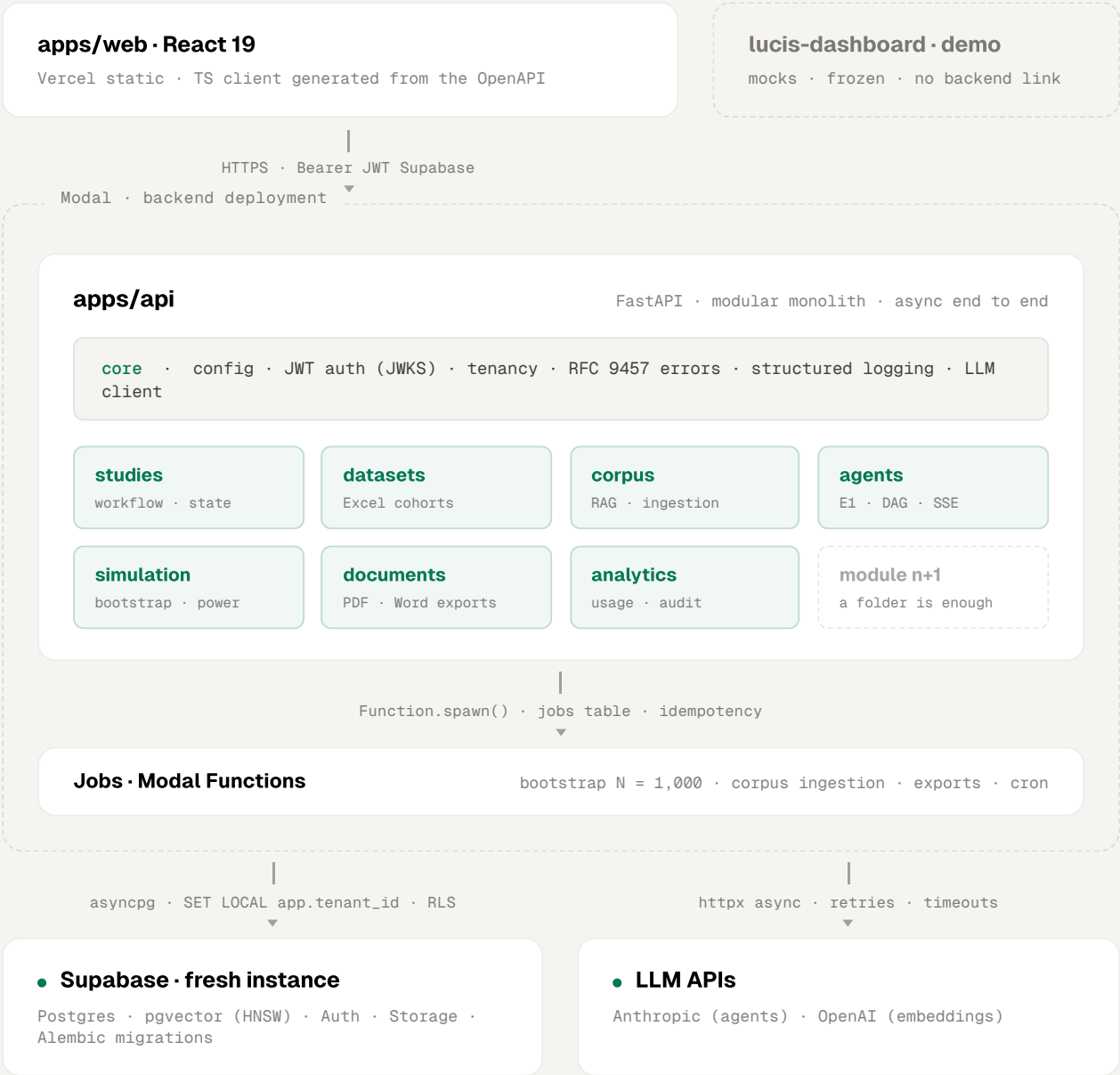
Replaces the Vercel functions and the offline scripts entirely. The current demo becomes an isolated front end artifact.

METHOD

An instrumented big bang: characterization tests built from real fixtures; the switch happens only once parity is proven.

A single door

The front end talks to a single door: the API. The API talks to a single database: Supabase. Heavy compute leaves the request path and becomes a Modal function.



Each module exposes `router` · `service` · `schemas` · `repo` and never imports the internals of another. The boundary is enforced by `import-linter` in CI. A module that outgrows the monolith extracts into a separate service without a rewrite: its public contract already exists.

Why Python

Not a preference: a structural fact. Augura's scientific core is written in Python and will not leave it. Any other backend language means two runtimes for life.

The domain core cannot move

ATE, ATT, LME, IPW, bootstrap, power analysis: scipy, statsmodels, sklearn. No credible equivalent in JavaScript; gonum (Go) is embryonic. Python unifies the API, the agents and the science.

AI is first class here

Native Anthropic SDK, agent outputs validated by Pydantic with a repair retry, LangSmith, embedding pipelines. The ecosystem where everything lands first.

One type contract, end to end

Pydantic v2 → OpenAPI → generated TypeScript client. Runtime validation, documentation and front end typing all come from a single source. The front end never writes a call by hand.

Modal is Python native

The entire API deploys as one ASGI decoration. Long jobs become functions: no Celery, no Redis, no queue to operate. Cron comes included, GPU within reach.

The Python speed debate is about a load profile Augura does not have. The backend is I/O bound: latency lives in the LLM calls, seconds at a time, and in the database, not in the interpreter. Heavy compute is vectorized numpy, C under the hood. Go would only win on massive HTTP throughput with light compute; Augura's B2B usage is nowhere near it.

WHAT MATTERS HERE

- One backend brain: API, agents and statistics in the same runtime.
- Agent outputs validated by schema, not parsed by hand.
- Long jobs with no queue infrastructure to operate.

WHAT WE CONCEDE, KNOWINGLY

- Node: shared types without generation, shorter cold starts.
- Go: a static binary, minimal memory footprint.
- Neither outweighs the first point.

Seven modules, one rule

Each module is an autonomous vertical slice: `router` · `service` · `schemas` · `repo`. A module never imports the internals of another, only the core and public interfaces. The rule is not a promise: CI breaks when it is violated.



core the foundation, not a module

Config that fails fast at boot, JWT verification (Supabase JWKS), tenant resolution, error hierarchy, structured logging, shared LLM client, domain events. Zero business logic; no imports into the modules.



studies

Study lifecycle, workflow state persisted and versioned in the database (sessionStorage retires), members and roles per study. Unlocks collaboration and work across devices.



datasets

Cohort uploads to Supabase Storage, pandas parsing and validation off the event loop, column profiling, variable mapping.



corpus

Tenants ingest their own documents: PDF, extraction, chunking, embeddings, into pgvector. Retrieval in direct SQL; one shared global corpus plus a private corpus per tenant.



agents

E1 profiling streamed over SSE, causal DAG, gap detection, variable check, all on one shared runtime: tool use loop, Pydantic outputs, the same NDJSON protocol the front end already knows.



simulation

Bootstrap on demand, as a Modal job on a dedicated scientific image, plus a synchronous analytical approximation calibrated per outcome. Manual precomputation disappears.



documents

Protocol and report generation: the full study state, an LLM and a template, rendered to PDF or Word, served by a signed Supabase Storage URL. Asynchronous, as a job.



analytics

Authenticated usage (soft auth retires), an audit trail fed by the event outbox, admin statistics, LLM cost and latency recorded per agent run.

Three paths, one discipline

Everything the backend does travels one of these three paths. Each has its latency budget, its failure mode and its contract.

Standard request

p95 < 300 ms excluding LLM

01 generated TS client — 02 JWT verified (JWKS) — 03 tenant resolved — 04 service —
05 repo + RLS — 06 Pydantic response

Every transaction opens with `SET LOCAL app.tenant_id`; the RLS policies build on it. Scoping is applied twice: in the repos, and by Postgres itself. A forgotten filter cannot leak data across tenants.

E1 agent · streaming

first token < 2 s · no 300 s ceiling

01 POST /agents/profiling/stream — 02 tool use loop — 03 corpus retrieval —
04 NDJSON events streamed — 05 study state persisted

Heartbeats while waiting; the stream always ends with a `done` or `error` event, never an endless spinner. Structured outputs validated by Pydantic, with one repair retry before a clean failure.

Heavy job · simulation, ingestion, export

202 immediately · tracked in jobs

01 POST /simulations — 02 insert **jobs** + `Function.spawn()` — 03 202 · job_id —
04 worker updates progress — 05 results read by the API

An idempotency key is required: a double click never launches two bootstraps. Failure means a `failed` status, a detailed error, a Sentry alert, a manual retry. The scientific image (numpy, scipy, statsmodels) stays separate from the API image.

Demo mode leaves the backend.
Production has a single code path.

Robust by construction

Robustness is not a code review: it is a set of rules the machine enforces. Three families, all checked automatically.

TYPING

- **pyright strict** in CI. No exceptions.
- Pydantic v2 at **every boundary**: requests, responses, settings, jobs, agent outputs.
- `NewType` for `TenantId`, `StudyId`, `JobId`.
- Any and type: `ignore` forbidden without a written justification.
- SQLAlchemy models never leave a module.

EXECUTION

- Async end to end; the event loop does **I/O only**.
- Anything beyond ~100 ms leaves the API: a thread or a Modal job.
- Deterministic agent responses cached: same inputs, served response.
- Keyset pagination; pgvector HNSW index.
- Budgets held: `p95 < 300 ms` excluding LLM, first SSE token `< 2 s`.

ERRORS

- `AppError` hierarchy; uniform **RFC 9457** responses.
- LLM down means an **explicit 503**, never an endless wait.
- Invalid structured output: one repair retry, then a clean failure.
- SSE: heartbeats, always ends with `done` or `error`.
- Failed jobs: detailed error, Sentry alert, manual retry.

CI BREAKS IF

- a module imports the internals of another (import-linter contract);
- `pyright strict`, `ruff` or any test fails (no overrides);
- the TypeScript client drifts from the OpenAPI contract (drift check);
- an RLS isolation test proves one tenant can read another.

These rules exist for a simple reason: a short team, augmented with agents, does not reread everything. **What the machine verifies needs no watching.**

Modal in production, a proven switch

Runtime

The API as `@modal.asgi_app()` on a light uv image, `min_containers = 1` in prod: no cold start for users. Jobs on a separate scientific image; cron comes included for recurring work.

Environments

Modal dev and prod: separate secrets, separate Supabase instances, versioned Alembic migrations. The front end stays on Vercel and points at the API domain.

CI chain

`ruff · pyright strict · import-linter · pytest · TS client drift check`, then `modal deploy` on main. Nothing ships unless everything is green.

Observability

structlog JSON with `request_id`, Sentry for errors, LangSmith for agent traces, LLM cost and latency per run in the database, visible in analytics.

01	Foundations	monorepo scaffold, quality gates, CI, healthcheck deployed on Modal dev
02	Core + studies	JWT auth, tenancy, first module, generated TS client; the migrated front end boots
03	datasets + corpus	cohort uploads, ingestion; existing corpus transferred with embeddings intact
04	agents	characterization first; E1 SSE ported last of the four
05	simulation + jobs	job infrastructure, bootstrap on demand, per outcome calibration
06	documents + analytics	PDF and Word generation, authenticated usage, audit
07	The switch	parity proven, production front end switched, demo frozen as an isolated artifact

**The switch is not a date.
It is a criterion.**

Golden tests green (the demo's real fixtures replayed against the new backend), e2e green, RLS isolation proven. Until everything is green, the old backend stays in service.

Augura

**The prototype proved the value.
The architecture proves it lasts.**

Specification: `docs/specs/2026-06-11-augura-backend-architecture-design.md`

Execution plan: `docs/plans/2026-06-11-phase-1-fondations.md`